

Mobisf License Analysis

Mobile Security Framework MobSF

LICENSES

GitHub License

Advantages of Using MobSF (GPL-3.0)

1. No Licensing Cost

MobSF is released under the **GPL-3.0 license**, meaning:

- No purchase or subscription fees
- Ideal for startups and early-stage businesses
- Reduces initial capital expenditure

2. Full Source Code Access

GPL-3.0 guarantees access to the complete source code:

- Enables deep customization of security features
- Allows integration into internal DevSecOps pipelines
- Supports advanced feature development based on business needs

3. Strong Mobile App Security Coverage

MobSF provides:

- Static and dynamic analysis
 - Malware detection
 - API, permission, and vulnerability analysis
- This enhances **application trust, compliance readiness, and customer confidence**.

4. Community-Driven Improvements

As an open-source framework:

- Continuous updates from the security community
- Faster discovery of vulnerabilities
- Shared best practices without vendor lock-in

5. Transparency and Auditability

GPL-licensed code ensures:

- No hidden functionality
- Easier security audits
- High suitability for regulated industries

6. Cross-Platform Support

MobSF supports Android and iOS applications:

- One framework for multiple platforms
- Reduced tooling fragmentation
- Easier team training and onboarding

Disadvantages of Using MobSF (GPL-3.0)

1. Mandatory Open-Sourcing of Derivative Work

The **most critical GPL-3.0 constraint**:

- If you modify MobSF **and distribute it**, you **must release your source code**
- This may conflict with proprietary business models

2. Not Suitable for Closed-Source Commercial Products

If your business plan requires:

- Proprietary security features
 - Closed-source SaaS distribution
- GPL-3.0 creates **legal and strategic limitations**

3. Legal Compliance Complexity

Using GPL-3.0 requires:

- Careful license compliance tracking
- Legal review before distribution
- Proper attribution and license inclusion

4. Limited Commercial Support

Unlike enterprise security tools:

- No official SLA
- Support depends on community or internal expertise
- May increase operational risk for mission-critical deployments

5. Integration Overhead

MobSF integration may require:

- Skilled security engineers
- Infrastructure setup and maintenance
- Additional CI/CD customization effort

6. Risk of License Contamination

If MobSF code is tightly coupled with your app:

- GPL obligations may extend to your entire codebase
- Improper isolation can expose proprietary components

Usage Strategy

To **avoid GPL-3.0 conflicts while gaining benefits**:

- Use MobSF as an **internal security testing tool**
- Do **not embed or distribute MobSF code** inside your app
- Keep MobSF isolated as a **separate service or pipeline**
- Distribute only **security results**, not modified code

This approach allows **commercial app development without forced open-sourcing**.

Summary Table

Aspect	Impact on Business
Cost	Very Low
Security Value	Very High
Commercial Flexibility	Limited
Legal Risk	Medium to High
Best Use Case	Internal security analysis tool

Final Recommendation

MobSF under GPL-3.0 is **excellent for internal app security analysis and feature validation**, but **not suitable for embedding into proprietary commercial apps** unless you are willing to open-source derivative code.

Other Licenses

1. Androguard (Apache License 2.0)

Business Advantages

- Strong Android static analysis (DEX, permissions, APIs)
- No copyleft risk (Apache 2.0 is business-friendly)
- Suitable for proprietary apps
- Can be embedded as a backend analysis engine

Business Disadvantages

- High technical complexity
- Requires Android reverse-engineering expertise
- Performance overhead on large APKs

Business Decision

USE

- Safe for commercial apps
- Best used as a **backend service**, not client-side

2. AdminLTE (MIT License)

Business Advantages

- Ready-made admin dashboard UI
- Fast development and clean UX
- No license restrictions
- Ideal for enterprise dashboards

Business Disadvantages

- Generic UI look if not customized
- Frontend only (no business logic)

Business Decision

USE

- Perfect for admin panels and reporting dashboards

3. APKInspector (GPL License)

Business Advantages

- Deep APK structure inspection
- Useful for debugging and forensic features

Business Disadvantages

- GPL copyleft risk
- Cannot be embedded into proprietary products
- Forces source disclosure if distributed

Business Decision

AVOID (Embedding)

USE ONLY INTERNALLY

- Keep as an **internal analysis tool**, not a shipped feature

4. ApkSigner / Apktool / Baksmali / Smali (Apache / BSD-style)

Business Advantages

- Essential for APK decoding and rebuilding
- Stable, widely used tools
- Permissive licenses (Apache/BSD)

Business Disadvantages

- Advanced usage may break app integrity
- Requires expert handling

Business Decision

USE

- Safe for backend automation
- Do not expose directly to end users

5. Class-Dump-Z (BSD-style License)

Business Advantages

- iOS binary analysis support
- Helpful for static iOS security checks

Business Disadvantages

- Limited iOS coverage compared to Android tools
- Niche use case

Business Decision

OPTIONAL

- Use only if iOS analysis is a business requirement

6. Droidmon (GPL-3.0)

Business Advantages

- Dynamic runtime behavior monitoring
- Detects malicious runtime actions

Business Disadvantages

- GPL-3.0 copyleft
- Cannot be bundled into proprietary software
- Requires emulator/rooted device

Business Decision

AVOID (Commercial Distribution)

USE INTERNALLY ONLY

- Suitable for internal security testing pipelines

7. Exodus-Core (GPL License)

Business Advantages

- Tracker and privacy detection
- Strong compliance and privacy insights

Business Disadvantages

- GPL restrictions
- Legal risk if embedded directly
- Tracker DB updates required

Business Decision

ISOLATE

- Use as a **separate scanning service**
- Expose results, not the code

8. Anti-Emulator / SSL Unpinning / Proxy Detection Modules (Mixed GPL)

Business Advantages

- High-value security features
- Detects evasion techniques
- Enterprise-grade security insight

Business Disadvantages

- GPL license risk
- High false-positive risk
- Can be legally sensitive in some regions

Business Decision

OPTIONAL / PREMIUM ONLY

- Best as enterprise or internal features
- Do not ship inside client apps

9. jQuery / Bootstrap / Jasny Bootstrap (MIT / Apache)

Business Advantages

- UI stability and rapid frontend development
- Fully commercial-safe
- Large ecosystem

Business Disadvantages

- Performance overhead if overused
- Older UI stack

Business Decision

USE

- Safe and reliable for dashboards

10. Cuckoo Sandbox Integration (GPL-3.0)

Business Advantages

- Advanced malware behavior analysis
- Enterprise-grade sandboxing

Business Disadvantages

- Heavy infrastructure cost
- GPL-3.0 license restrictions
- High operational complexity

Business Decision

DO NOT EMBED

- Use as an **external service only**

11. Androguard (Apache 2.0)

Advantages

- Deep Android static analysis (DEX, permissions, APIs)
- Commercial-safe license
- High value for security features

Disadvantages

- Requires expert knowledge
- Heavy computation for large apps

Decision: USE

12. JADX (Apache 2.0)

Advantages

- Java source reconstruction
- Improves transparency and debugging features
- Safe for proprietary products

Disadvantages

- Decompiled output may be inaccurate
- Performance overhead

Decision: USE

13. APKTool / ApkSigner (Apache 2.0)

Advantages

- Industry-standard APK decoding and signing
- Enables automation features
- No license risk

Disadvantages

- Can break app integrity if misused

Decision: USE

14. Smali / Baksmali (BSD-style)

Advantages

- Low-level bytecode analysis
- Highly flexible

Disadvantages

- Steep learning curve
- Not user-friendly

Decision: USE (Backend only)

15. AdminLTE (MIT)

Advantages

- Fast admin UI development
- Clean dashboards
- Zero restrictions

Disadvantages

- Generic appearance without customization

Decision: USE

16. jQuery / Bootstrap / Jasny Bootstrap (MIT / Apache)

Advantages

- Stable UI frameworks
- Large ecosystem

Disadvantages

- Slight performance overhead

Decision: USE

17. Class-Dump-Z (BSD)

Advantages

- iOS binary inspection
- Complements Android analysis

Disadvantages

- Limited depth vs Android tools

Decision: OPTIONAL

18. vd2svg (Apache 2.0)

Advantages

- Visualizes vector drawables
- Useful for reporting features

Disadvantages

- Non-core business value

Decision: OPTIONAL

19. Exodus-Core (GPL)

Advantages

- Tracker and privacy detection
- Strong compliance value

Disadvantages

- GPL copyleft risk
- Cannot embed in closed-source apps

Decision: ISOLATE (External service)

20. APKInspector (GPL)

Advantages

- Deep APK forensic inspection

Disadvantages

- GPL forces source disclosure if distributed

Decision: INTERNAL ONLY

21. Droidmon (GPL-3.0)

Advantages

- Runtime behavior monitoring
- Malware detection

Disadvantages

- Requires rooted/emulated devices
- GPL-3.0 legal risk

Decision: INTERNAL ONLY

22. Cuckoo Sandbox (GPL-3.0)

Advantages

- Enterprise-grade dynamic analysis

Disadvantages

- Infrastructure heavy
- Strong copyleft

Decision: EXTERNAL ONLY

23. Frida (wxWindows Library License)

Advantages

- Powerful runtime instrumentation
- Advanced security research features

Disadvantages

- Can be misused
- Legal sensitivity in some markets

Decision: OPTIONAL / INTERNAL

24. F-Secure Frida Scripts (BSD-3 Clause)

Advantages

- Ready-made security scripts
- Commercial-safe

Disadvantages

- Narrow use cases

Decision: USE (Optional feature)

25. RootCloak (Apache 2.0)

Advantages

- Root detection bypass testing
- Useful for threat modeling

Disadvantages

- Not needed for all users

Decision: OPTIONAL

26. iOS Binary Analysis Rules (BSD)

Advantages

- Rule-based iOS inspection
- Lightweight

Disadvantages

- Rule maintenance required

Decision: USE (iOS-focused apps)

27. Maltrail Blacklist (MIT)

Advantages

- Network threat intelligence
- Easy integration

Disadvantages

- Requires frequent updates

Decision: USE

28. IP2Location LITE Data (Custom Data License)

Advantages

- Geo-location enrichment
- Business analytics value

Disadvantages

- **Redistribution prohibited**
- Attribution mandatory

Decision: USE WITH COMPLIANCE ONLY

18 LICENSES BUSINESS DECISION SUMMARY

#	Component	License	Commercial Risk	Decision
1	Androguard	Apache 2.0	Low	USE
2	JADX	Apache 2.0	Low	USE
3	APKTool / ApkSigner	Apache 2.0	Low	USE
4	Smali / Baksmali	BSD	Low	USE
5	AdminLTE	MIT	Low	USE

6	jQuery / Bootstrap	MIT / Apache	Low	USE
7	Class-Dump-Z	BSD	Low	OPTIONAL
8	vd2svg	Apache 2.0	Low	OPTIONAL
9	Exodus-Core	GPL	High	ISOLATE
10	APKInspector	GPL	High	INTERNAL ONLY
11	Droidmon	GPL-3.0	High	INTERNAL ONLY
12	Cuckoo Sandbox	GPL-3.0	High	EXTERNAL ONLY
13	Frida	wxWindows	Medium	OPTIONAL
14	F-Secure Frida Scripts	BSD-3	Low	USE
15	RootCloak	Apache 2.0	Low	OPTIONAL
16	iOS Binary Rules	BSD	Low	USE
17	Maltrail Blacklist	MIT	Low	USE
18	IP2Location LITE	Custom	Medium	USE (with attribution)

Key Business Rule

- **Permissive license** → Can embed
- **Permissive licenses** → Embed freely
- **GPL license** → Do NOT embed
- **GPL licenses** → Never embed in proprietary apps
- **GPL tools** → Use internally or as isolated services
- **Data licenses** → Follow attribution and redistribution limits

Truecaller App Statistical Analysis

 truecaller-caller-id-and-spam-call-blocker-15-28-7 apk.pdf

 Virus Total truecaller-caller-id-and-spam-call-blocker-15-28-7.pdf

 metadefender-results-hash-20857774a819c1a88140244fa5bc10d7678f6...

APK Used

Report of Mobile Security Framework MobSF

[Mobile-Security-Framework-MobSF](#)

1. Overview and Purpose

The **Mobile Security Framework (MobSF)** (GitHub: [\[MobSF/Mobile-Security-Framework-MobSF\]](#)) is a well-established open-source security research platform designed for mobile application security testing across **Android, iOS, and Windows** platforms. Its primary use is to automate security assessment tasks that would otherwise require multiple standalone tools and significant manual effort.

MobSF is widely used by developers, mobile security researchers, penetration testers, and DevSecOps teams. It supports both **static and dynamic analysis**, including malware assessment and privacy analysis.

Key Attributes (from GitHub and ecosystem):

- **Stars:** ~20,000+ (indicative of a large user base).
- **Forks:** ~3,500+ (active open-source involvement).
- **License:** GPL-3.0.
- **Written primarily in:** Python and JavaScript (with HTML/CSS for frontend).

2. Core Functionality and Features

The MobSF framework is structured to perform multiple types of mobile application security assessments:

2.1 Static Analysis (SAST)

Static analysis inspects application binaries or source code **WITHOUT** executing the application.

MobSF supports analysis of:

- APK (Android packages)
- AAB (Android bundle)
- IPA (iOS packages)
- APPX (Windows mobile)

- Zipped source code repositories

This includes:

- Manifest inspection (permissions, exported components)
- Code decompilation
- Hard-coded secrets detection (API keys, passwords)
- Cryptography and insecure configuration checks
- Third-party library and dependency scanning

2.2 Dynamic Analysis (DAST and Runtime)

Dynamic analysis entails observing app behavior during execution.

MobSF offers:

- Runtime instrumentation and monitoring
 - Network traffic analysis
 - Instrumented testing on emulator environments
- Note:* Real device dynamic testing is limited and not fully supported in all cases.

2.3 Malware and Privacy Analysis

MobSF includes capabilities to identify:

- Malicious behaviors and suspicious constructs inside binaries
- Privacy-related concerns (data leakage)
- Abnormal API or system call usage

2.4 CI/CD and DevSecOps Integration

MobSF provides:

- RESTful APIs
 - CLI tooling
 - Hooks to integrate into automated build/test pipelines
- This enables security testing to become part of automated development flows, such as Jenkins, GitLab CI, GitHub Actions, etc.

3. Architecture and Code Structure

While the GitHub repo is large and complex (thousands of commits and many directories), here's a high-level breakdown:

3.1 Repository Structure (core components)

- **mobsf/** – Main source code (core static and dynamic engine)
 - **docker/** – Docker setup and configuration
 - **scripts/** – Utility scripts (helper tools)
 - **setup.sh, run.sh, setup.bat, run.bat** – Installation and start scripts
- This multi-OS structure indicates support for both Unix-like and Windows environments.

3.2 Technology Stack

- **Backend:** Python (analysis engine)
 - **Frontend/UI:** JavaScript + HTML/CSS (web interface)
 - **DevOps:** Docker, CLI tooling
- This mix supports ease of installation and usage in multiple environments.

4. Security Posture and Considerations

Being a security testing tool itself, MobSF must be kept **up-to-date** due to potential vulnerabilities:

Documented Vulnerabilities (from CVE history)

MobSF has had issues in the past that were addressed:

- **Stored XSS vulnerability**, fixed in version 4.2.9.
- **Zip extraction Zip Slip flaw** before version 4.0.7.
- **ZIP upload processing risks**, documented in CVE-2025-46730.

These issues illustrate both the power and potential risk of an analysis framework running untrusted inputs. Users should run MobSF in isolated environments (e.g., containers).

5. Usage and Installation

5.1 Typical Install Steps

- Clone the repo:
`git clone`
<https://github.com/MobSF/Mobile-Security-Framework-MobSF.git>
- Run setup scripts:

- On Linux/Mac: `./setup.sh`
 - On Windows: `setup.bat`
 - Start the service:
 - On Linux/Mac: `./run.sh`
 - On Windows: `run.bat`
- These steps are standard in community documentation.

5.2 Docker Option

There's an official Docker image:

```
docker pull  
opensecurity/mobile-security-framework-mobsf:latest  
docker run -it --rm -p 8000:8000 ...
```

This simplifies deployment and reduces dependency pain points on hosts.

6. Integration and Ecosystem

MobSF is part of a larger ecosystem of security tools:

- **MobSFScan** – A static analysis plugin/tool for use with MobSF rule sets.
- **httptools** – HTTP request capture and replay tool related to MobSF workflows.

There are also associated educational materials such as:

- Presentations (BlackHat, OWASP)
 - Online courses for automated mobile assessment
- These reinforce MobSF's position as a "learning" as well as "tooling" platform.

7. Strengths, Limitations, and Use Cases

7.1 Strengths

- **All-in-one capability:** combines SAST, DAST, malware, and privacy analysis in one tool.
- **Multi-platform:** supports Android/iOS/Windows.
- **Open Source & Free:** community-driven.
- **CI/CD integration:** helps automate security testing.

7.2 Limitations

- **Dynamic analysis limitations:** real-device support is limited compared to emulators.
- **Surface area risk:** receives untrusted files; must be sandboxed.
- **Not a replacement** for full professional security lab tools in large enterprise environments.

7.3 Typical Use Case Scenarios

- Mobile app security audit
- Penetration testing reports
- Early defect and vulnerability detection in DevOps pipelines
- Security research and education

8. Conclusion

The **MobSF** project is a mature, open-source, and widely used mobile application security testing framework emphasizing ease of use and automation. It bridges static and dynamic analysis and offers capabilities that are essential to modern DevSecOps and security research workflows.

The repository is **actively maintained**, has **strong community adoption**, and has **multiple integration points** that support embedding deep security checks into development lifecycles.

.jar Files Analysis

MobSF .jar files

- 468b494d5a9e1814781e11bb945b95a2ce67065294ad75289a4ba4b0bad028f2_0.jar
- 468b494d5a9e1814781e11bb945b95a2ce67065294ad75289a4ba4b0bad028f2_1.jar
- 468b494d5a9e1814781e11bb945b95a2ce67065294ad75289a4ba4b0bad028f2_2.jar
- 468b494d5a9e1814781e11bb945b95a2ce67065294ad75289a4ba4b0bad028f2_3.jar

Section 1: Android API Reference JARs

Used only to understand Android system functions during analysis.

FIRST 4 JAR files initially,

These were **Android framework stub JARs** (very different purpose).

What those 4 JARs are (simple)

They are **reference libraries** that list Android system functions, **not tools**.

Table for the first 4 JARs

JAR Files (First Four)	Purpose (Simple)
framework_*.jar (4 files)	Provide Android system function definitions for analysis

One-line explanation you can use

- These JAR files contain Android system API references used to understand which system functions an app calls during static analysis.

Mobile Security Framework MobSF performs **static analysis** of Android apps (.APK/.AAB) by:

- Decompiling DEX bytecode
- Resolving Android framework APIs
- Mapping permissions, sources/sinks, and dangerous calls
- Running rule-based and signature-based security checks

To do this **without a full Android SDK**, MobSF ships **special Android API JARs**.

What These .jar Files Are

Key Observation on Files

Each JAR contains **only one file**:

classes.dex

That means:

- These are **DEX-based JARs**
- They are **NOT normal Java libraries**
- They are **Android framework stubs**, not application logic

What These JAR Files Do

1. Android API Stub Libraries (Core Purpose)

These JARs are **Android framework stub JARs**, functionally equivalent to:

android.jar

They contain:

- Class names
- Method signatures
- Field definitions
- Resource IDs

They **DO NOT** contain real implementations.

👉 Their only job is to let MobSF:

- Resolve Android API calls
- Understand method ownership
- Perform static call-graph and taint analysis

2. Why They Contain **classes.dex**

MobSF analyzes **DEX bytecode**, not JVM bytecode.

So instead of:

java/lang/String.class

They contain:

classes.dex

This ensures:

- Compatibility with APK/D8/Dalvik analysis
- Accurate reflection of Android runtime APIs

Why There Are Multiple JARs

Each of your four JARs corresponds to a **different Android API level**.

JAR	Represents	Why Needed
_0.jar	Older Android APIs	Legacy app support
_1.jar	Mid-range APIs	Common production apps
_2.jar	Newer APIs	Modern SDK features
_3.jar	Latest APIs	Android 12+ / scoped storage

MobSF automatically picks the correct one based on:

uses-sdk → **targetSdkVersion**

What They Are Used For in MobSF (Pipeline View)

Static Analysis Flow

APK

- └ **classes.dex** (app)
- └ **framework_stub.jar** (these JARs)
- └ **MobSF Analyzer**

MobSF uses these JARs to:

✓ Resolve calls like:

android.telephony.SmsManager.sendTextMessage()

✓ Detect:

- SMS abuse

- Clipboard access
- Location tracking
- Crypto misuse
- Insecure storage APIs

✓ Map APIs → permissions → attack surface

What They DO NOT Do

- ✗ No executable logic
- ✗ No network calls
- ✗ No telemetry
- ✗ No runtime behavior
- ✗ No malware payloads

They are **passive metadata libraries**.

Should You Use These in Your Own App?

DO NOT Use If:

- You are building a **production Android app**
- You expect runtime behavior
- You plan to ship them in APK/AAB

USE IF:

- You are building:
 - A **static analyzer**
 - A **security scanner**
 - A **reverse-engineering tool**
 - An **APK inspection backend**
- You need Android API resolution **without SDK**

Business (App Decision)

Use Case	Recommendation
Mobile security product	✓ Strongly recommended
App store app	✗ Never
Malware detection engine	✓ Required

Dev tooling (CI security scan)	✓ Required
Runtime SDK	✗ Invalid

Final Verdict

These **.jar** files are Android framework stub libraries used exclusively for static analysis.

They enable MobSF to **understand Android APIs without executing anything**.

They are:

- ✓ Safe
- ✓ Non-executable
- ✓ Essential for static security analysis
- ✓ NOT suitable for runtime app development

■ apksigner.jar ■ apktool_2.10.0.jar ■ baksmali-3.0.8-dev-fat.jar ■ base.jar
■ bundletool-all-1.17.2.jar

Section 2: Analysis & Build Tool JARs

- Used to decode, inspect, and verify Android applications.

1. apksigner.jar

What it does

- Verifies **APK cryptographic signatures**
- Supports:
 - v1 (JAR signing)
 - v2 / v3 / v4 (APK Signature Scheme)
- Extracts:
 - Certificate info
 - Signing algorithms
 - Debug vs release signing

How MobSF uses it

- Detects:
 - Debug-signed APKs
 - Weak hash algorithms (SHA1, MD5)
 - Multiple signers / tampering

- Part of **APK integrity & trust analysis**

Business POV

Aspect	Verdict
Security analysis	✓ Essential
Runtime app use	✗ Never
CI security checks	✓ Recommended

2.apktool_2.10.0.jar

What it does

- **Decodes APKs** into:
 - `AndroidManifest.xml` (readable)
 - `res/` resources
 - `smali/` code
- Rebuilds APKs (not used by MobSF)

How MobSF uses it

- Extracts:
 - Permissions
 - Components (activities, services, receivers)
 - Hardcoded values in resources
- Enables **manifest & resource security rules**

Business POV

Aspect	Verdict
Static analysis	✓ Core dependency
Reverse engineering	✓ Yes
App distribution	✗ No

3. baksmali-3.0.8-dev-fat.jar

What it does

Converts:

classes.dex → smali code

- Smali = human-readable Dalvik bytecode

How MobSF uses it

- Detects:
 - Dangerous APIs
 - Hardcoded secrets
 - Crypto misuse
 - Network & file operations
- Enables **deep bytecode inspection**

Business POV

Aspect	Verdict
Malware analysis	✓ Mandatory
IP protection tools	✓ Useful
Runtime apps	✗ Invalid

4.base.jar

What it does

- **Android framework stub JAR**
- Contains:
 - Android class definitions
 - Method signatures
- No real implementation

How MobSF uses it

Resolves calls like:

android.location.LocationManager.getLastKnownLocation()

- Needed for:
 - API mapping

- Permission inference
- Source–sink analysis

Business POV

Aspect	Verdict
Static analyzers	✓ Required
Android apps	✗ Never ship
SDK replacement	✗ No

5.bundletool-all-1.17.2.jar

What it does

- Processes **Android App Bundles (.aab)**

Converts:

- **.aab** → **split APKs**

How MobSF uses it

- Allows scanning of:
 - Play Store–ready apps
- Ensures:
 - Feature module analysis
 - Dynamic delivery inspection

Business POV

Aspect	Verdict
Modern app scanning	✓ Required
App runtime	✗ Not applicable
Play Store tooling	⚠ Internal only

6.gradle-wrapper.jar

What it does

- Bootstraps **Gradle builds**
- Downloads correct Gradle version automatically

How MobSF uses it

- Used only for:
 - Building MobSF components
 - Dev / CI pipelines

Business POV

Aspect	Verdict
Build systems	✓ Standard
Production APK	✗ Never
Security analysis	✗ Not involved

7.R.jar

What it does

- Contains compiled **Android resource IDs**

Equivalent to:

- **R.class** → **resource mapping**

How MobSF uses it

- Resolves:
 - Resource references
 - UI strings
 - Layout-linked logic
- Helps track:
 - Hardcoded URLs
 - API keys in resources

Business POV

Aspect	Verdict
--------	---------

Static inspection	✓ Helpful
App packaging	✗ No
Resource analysis	✓ Yes

8.vd2svg-0.4.4.jar

What it does

Converts:

- **VectorDrawable** → **SVG**
- Used for graphical assets

How MobSF uses it

- Renders vector icons in reports
- Improves **HTML security report readability**

Business POV

Aspect	Verdict
UI reporting	✓ Optional
Security logic	✗ None
App runtime	✗ No

Summary Table (Decision Matrix)

JAR	Category	Needed for Static Analysis	Safe to Ship in App
apksigner.jar	Crypto / Integrity	✓	✗
apktool.jar	Decompilation	✓	✗
baksmali.jar	Bytecode analysis	✓	✗
base.jar	Android API stubs	✓	✗

bundletool.jar	AAB processing	✓	✗
gradle-wrapper.jar	Build tooling	✗	✗
R.jar	Resource mapping	✓	✗
vd2svg.jar	Reporting	⚠ Optional	✗

Final Verdict (Business)

All these JARs are **tooling dependencies**, not libraries for runtime apps.

They are:

- ✓ Safe
- ✓ Offline
- ✓ Non-executing at runtime
- ✓ Essential for **static Android security analysis**

They **must never be bundled into a production APK**, but are **ideal for**:

- Mobile security products
- CI security scanners
- App vetting platforms
- Malware detection engines

App Copy Can or Not

Can We Use MobSF Logic for Our Business?

Context

The project **Mobile Security Framework MobSF** is released under the **GPL-3.0 license**.

You asked:

Can my business use this code by copying only the logic (not the code), possibly rewriting it in another language, to avoid license or copyright issues?

Short Answer

✓ Yes, you can use the *ideas and logic*

✗ No, you cannot copy the actual code or structure

If done correctly, **your business can legally build a similar system** without violating MobSF's license.

The Key Difference You Must Understand

1. What is NOT protected

These are **free to use**:

- General ideas
- Algorithms (at a high level)
- Security concepts
- Workflows
- “What to check” logic (e.g., check permissions, detect hardcoded secrets)

➡ Example (allowed):

“Scan an Android app, extract permissions, and flag dangerous ones.”

2. What IS protected

These are **NOT allowed to copy**:

- Source code (Python, Java, shell, etc.)
- Function names

- File/module structure
- Exact rule definitions
- Regex patterns copied line-by-line
- Database schemas copied directly

➡ Example (not allowed):

```
def check_debuggable_manifest(manifest):
    ...
```

or copying the same rule list in the same format.

What “Copying the Logic” LEGALLY Means

✓ Allowed (Safe Path)

You can:

- Read MobSF to **understand how it works**
- Write your **own implementation from scratch**
- Use:
 - Different languages (Go, Rust, Java, Python, etc.)
 - Different architecture
 - Different rule definitions
- Design your **own rule engine**

This is called:

Clean-room reimplementation

✗ Not Allowed (Risky / Illegal)

You cannot:

- Translate Python → Java line by line [CodeConvert AI](#)
- Rename variables but keep same structure
- Reuse MobSF rule files with minor edits
- Keep same folder layout and logic flow

Changing language **alone** does NOT make it legal.

GPL-3.0 in Very Simple Terms

GPL-3.0 says:

“If you copy or modify this code, your product must also be open source under GPL-3.0.”

So businesses usually choose **not to copy the code**, but instead:

- ✓ Re-implement the idea
- ✓ Keep their product closed-source
- ✓ Avoid GPL obligations

Business-Safe Strategy (Recommended)

Step 1: Use MobSF as Learning Material

- Understand:
 - What checks are done
 - Why they matter
 - What outputs are useful

Step 2: Design Your Own System

- Write a **new architecture**
- New naming
- New rule formats
- New output format

Step 3: Implement Independently

- Use your own:
 - Logic flow
 - Scoring system
 - Detection rules
- Document that it is:
“Inspired by common mobile security practices”

⚠ Do **not** say “based on MobSF code”.

Simple Terms (Very Important)

Think of MobSF like a **cookbook**:

- ❌ You cannot copy the recipe word-for-word and sell it
- ✅ You *can* cook the same dish using your own recipe

Can You Implement It in Another Language?

Yes — **this actually helps**, but only if:

- ✓ You don't mirror the code structure
- ✓ You don't copy rules verbatim
- ✓ You design independently

Language change alone is **not enough**, but it is **a good step**.

What You Can Safely Do

You can use this text:

This system is an independent implementation inspired by publicly known mobile application security analysis techniques. No source code from third-party GPL-licensed projects has been copied or reused.

Final Verdict (Humanised)

- ✓ You **can** use MobSF ideas and concepts
- ✗ You **cannot** copy or translate its code
- ✓ Re-implementation from scratch is legal
- ✓ Your business can remain closed-source
- ⚠ Be careful with rule definitions and structure

1. What We Want to Do

Build our own mobile security analysis product by **learning from MobSF**, but **not copying its code**, possibly implementing it in another language.

2. Clear Yes / No Decision Table

Area	What MobSF Provides	Can We Use it for Business?	Decision	Reason (Simple)
High-level ideas	App scanning, permission checks, API risk analysis	✓ Yes	APPROVED	Ideas and concepts are not copyrighted
Security logic (conceptual)	What to check and why	✓ Yes	APPROVED	This is general security knowledge

Exact source code	Python, Java, scripts	✗ No	REJECTED	GPL-3.0 forbids closed-source reuse
Code structure / flow	Same modules, same pipeline	✗ No	REJECTED	Considered derivative work
Rule files copied verbatim	Regex, hardcoded patterns	✗ No	REJECTED	Direct copying = license violation
Rewriting logic from scratch	Our own implementation	✓ Yes	APPROVED	Clean-room implementation
Implementing in another language	Go / Rust / Java / Python	✓ Yes	APPROVED	Helps avoid code similarity
Using MobSF tool JARs	apktool, bak smali, etc.	⚠ Yes (separately)	CONDITIONAL	Each tool has its own license
Shipping MobSF code in product	Bundling code	✗ No	REJECTED	Forces GPL on entire product
Commercial closed-source product	Our own scanner	✓ Yes	APPROVED	If no MobSF code is reused

3. Tool-Level Decision (Very Important)

Tool / Component	Can Business Use It?	Conditions
apktool	✓ Yes	Follow its Apache 2.0 license
baksmali	✓ Yes	Follow its BSD-style license
apksigner	✓ Yes	Android SDK license
bundletool	✓ Yes	Apache 2.0

MobSF Python code	✗ No	GPL-3.0 restriction
MobSF rules & regex	✗ No	Considered code/content

👉 Tools are safe

👉 MobSF orchestration code is not

4. Safe Business Implementation Strategy (Approved)

What We SHOULD Do

- Study MobSF **only to understand concepts**
- Design our **own architecture**
- Write **new rules**
- Use **our own naming, structure, scoring**
- Implement in **another language or fresh Python**
- Clearly document independent implementation

What We MUST NOT Do

- Copy-paste code
- Translate line-by-line
- Reuse rule definitions
- Keep same folder/module layout

5. Risk Assessment Table

Risk Type	Risk Level	Mitigation
License violation	🔴 High if code copied	Clean-room rewrite
Legal exposure	🔴 High if derivative	Independent design
Business IP risk	🟢 Low if reimplemented	Proper documentation
Technical feasibility	🟢 Low risk	Proven by MobSF concept
Market differentiation	🟡 Medium	Add unique features

6. Final Business Verdict (Clear & Humanised)

We can legally and safely build a commercial product inspired by MobSF, provided we do not copy or translate its code, rules, or structure.

Re-implementing the logic from scratch, even in another language, is allowed and recommended.

Using third-party tools is acceptable if their individual licenses are respected.

7. One-Line Executive Summary

Decision: APPROVED — Proceed with a clean-room reimplementation inspired by MobSF concepts, without reusing GPL-licensed code or rules.

Features of Statistic Analysis

Features of Statistic Analysis

1. Static Analyzer

- 1.1 Information
- 1.2 Scan Options
- 1.3 Signer Certificate
- 1.4 Permissions
- 1.5 Android API
- 1.6 Browsable Activities

2. Security Analysis

- 2.1 Network Security
- 2.2 Certificate Analysis
- 2.3 Manifest Analysis
- 2.4 Code Analysis
- 2.5 Binary Analysis
- 2.6 NIAP Analysis
- 2.7 File Analysis
- 2.8 Firebase Analysis

3. Malware Analysis

- 3.1 Malware Analysis
- 3.2 Malware Lookup
- 3.3 APKiD Analysis
- 3.4 Behaviour Analysis
- 3.5 Abused Permissions

4. Infrastructure & Threat Intelligence

- 4.1 Server Locations
- 4.2 Domain Malware Check

5. Reconnaissance

- 5.1 URLs
- 5.2 Emails
- 5.3 Trackers

5.4 Hardcoded Secrets

5.5 Strings

6. Application Components

6.1 Components

6.2 Activities

6.3 Services

6.4 Receivers

6.5 Providers

7. Dependencies & Inventory

7.1 Libraries

7.2 SBOM (Software Bill of Materials)

8. File System Analysis

8.1 Files

Features With Explanation

1. Static Analyzer

- Analyze the app **without running it**
- Works by **reading files**, not executing the app.

1.1 Information

Logic: Collect basic metadata about the app.

Example: App name = "SampleApp", version = 1.2.3, target SDK = 33.

Concept: Understand *what the app claims to be* before analyzing behavior.

How it finds:

- Reads the app metadata file (similar to reading a JSON or YAML config).
- Extracts key-value pairs.

How it marks result:

- Displays extracted values as informational output (not a warning).

1.2 Scan Options

Logic: Control how deep the analysis should go.

Example: Full scan enabled to analyze code, files, and network usage.

Concept: Different situations require different analysis depth.

How it finds:

- Scan options are user inputs that control which modules run.

How it marks result:

- Enables/disables checks; no risk marking.

1.3 Signer Certificate

Logic: Identify the certificate used to sign the app.

Example: App signed with a debug certificate instead of a release certificate.

Concept: App authenticity depends on who signed it.

How it finds:

- Reads cryptographic signature from the app package.
- Parses certificate fields (issuer, validity, algorithm).

How it marks result:

- If debug certificate → mark as **warning**
- If weak algorithm → mark as **high risk**

1.4 Permissions

Logic: List all permissions requested by the app.

Example: App requests access to contacts and device location.

Concept: Permissions indicate what sensitive data the app can access.

How it finds:

- Parses the permission list from configuration file.
- Compares permissions against a predefined “dangerous permissions” list.

How it marks result:

- Dangerous permission found → **flag**
- Excessive permissions → **risk score increased**

1.5 Android API

Logic: Detects Android system APIs used by the app.

Example: App uses SMS API to send text messages.

Concept: APIs reveal the app's technical capabilities.

How it finds:

- Scans code instructions for known system API names.
- Matches API names against a sensitive-API list.

How it marks result:

- Sensitive API used → **informational or warning**, depending on API type

1.6 Browsable Activities

Logic: Identify app screens that can be launched externally.

Example: Login screen opens directly from a web link.

Concept: Externally accessible screens increase security risk.

How it finds:

- Reads screen definitions and checks exposure flags.
- Detects whether screens allow external access.

How it marks result:

- Externally accessible screen → **medium risk**

2. Security Analysis

- Detect security weaknesses and unsafe configurations
- Rule-based detection of insecure patterns.

2.1 Network Security

Logic: Analyze how the app communicates over the network.

Example: App sends user data using unencrypted HTTP.

Concept: Insecure communication exposes sensitive data.

How it finds:

- Searches configuration and code for network protocol usage.
- Look for plain-text protocols.

How it marks result:

- HTTP usage → **high risk**

- HTTPS enforced → **pass**

2.2 Certificate Analysis

Logic: Check how the app validates server certificates.

Example: App accepts any SSL certificate without validation.

Concept: Proper certificate validation prevents interception attacks.

How it finds:

- Searches code for certificate validation logic.
- Detects patterns that bypass validation.

How it marks result:

- Validation bypass → **critical risk**

2.3 Manifest Analysis

Logic: Inspect app configuration settings.

Example: Application is marked as debuggable in production.

Concept: Misconfigured settings weaken app security.

How it finds:

- Reads configuration flags.
- Compares values against secure defaults.

How it marks result:

- Unsafe flag → **warning or high risk**

2.4 Code Analysis

Logic: Examine code logic for insecure practices.

Example: Password stored directly in source code.

Concept: Insecure coding leads to data compromise.

How it finds:

- Converts compiled code into readable instructions.
- Searches for insecure keywords and patterns.

How it marks result:

- Match found → **issue with severity based on pattern**

2.5 Binary Analysis

Logic: Inspect compiled binaries for risky patterns.

Example: Native library uses unsafe memory functions.

Concept: Low-level code can introduce critical vulnerabilities.

How it finds:

- Inspects native binary files.
- Looks for unsafe function calls.

How it marks result:

- Unsafe function detected → **high risk**

2.6 NIAP Analysis

Logic: Check security compliance against standards.

Example: App fails secure storage requirements.

Concept: Standards ensure baseline security quality.

How it finds:

- Maps detected behaviors against compliance rules.
- Checks whether required security controls exist.

How it marks result:

- Missing control → **compliance failure**

2.7 File Analysis

Logic: Analyze files bundled within the app.

Example: Configuration file contains an API key.

Concept: Sensitive data should not be stored in plain files.

How it finds:

- Iterates over files inside the app.
- Applies pattern matching (keys, secrets, configs).

How it marks result:

- Sensitive content found → **warning**

2.8 Firebase Analysis

Logic: Inspect cloud database configuration.

Example: Firebase database allows public read access.

Concept: Cloud misconfiguration can leak user data.

How it finds:

- Reads cloud configuration files.
- Checks access rules.

How it marks result:

- Public access detected → **critical risk**

3. Malware Analysis

- Determine whether the app behaves maliciously
- Pattern + reputation based detection.

3.1 Malware Analysis

Logic: Detect known malicious behavior patterns.

Example: App silently sends premium-rate SMS messages.

Concept: Malicious behavior often follows known patterns.

How it finds:

- Matches behaviors against known malware behavior rules.

How it marks result:

- Rule match → **malware indicator**

3.2 Malware Lookup

Logic: Compare app fingerprint against malware databases.

Example: App hash matches a known malware sample.

Concept: Known threats can be identified via signatures.

How it finds:

- Calculates app fingerprint (hash).
- Queries malware signature database.

How it marks result:

- Match found → **confirmed malware**

3.3 APKiD Analysis

Logic: Detect packing and obfuscation techniques.

Example: App uses heavy code obfuscation to hide logic.

Concept: Obfuscation can be used to evade analysis.

How it finds:

- Scans app structure for packing/obfuscation markers.

How it marks result:

- Obfuscation detected → **suspicious**

3.4 Behaviour Analysis

Logic: Infer intent by combining multiple behaviors.

Example: App reads contacts and sends them to a remote server.

Concept: Combined actions reveal true intent.

How it finds:

- Correlates multiple findings.
- Looks at sequences, not single events.

How it marks result:

- Risky combination → **behavior warning**

3.5 Abused Permissions

Logic: Identify misuse of granted permissions.

Example: Location permission used continuously in background.

Concept: Legitimate permissions can still be abused.

How it finds:

- Compares permissions vs actual API usage.
- Detects unnecessary or excessive use.

How it marks result:

- Misuse → **medium to high risk**

4. Infrastructure & Threat Intelligence

4.1 Server Locations

Logic: Identify geographic location of backend servers.

Example: App connects to servers hosted in high-risk regions.

Concept: Server location can indicate compliance and risk.

How it finds:

- Extracts server IPs/domains.
- Uses IP-to-location mapping.

How it marks result:

- High-risk region → **informational or warning**

4.2 Domain Malware Check

Logic: Check contacted domains against threat lists.

Example: App communicates with a known malicious domain.

Concept: Infrastructure reputation affects trust.

How it finds:

- Compares extracted domains with threat lists.

How it marks result:

- Malicious domain → **critical risk**

5. Reconnaissance

- Extract potentially sensitive information
- Data extraction using pattern matching.

5.1 URLs

Logic: Extract hardcoded web links.

Example: Hidden admin panel URL found in code.

Concept: Hardcoded endpoints may expose internal systems.

How it finds:

- Regex search for URL patterns in files.

How it marks result:

- URL listed as finding (not always risk).

5.2 Emails

Logic: Extract embedded email addresses.

Example: Developer email found in resources.

Concept: Exposed emails can be misused.

How it finds:

- Regex search for email patterns.

How it marks result:

- Email exposed → **informational**

5.3 Trackers

Logic: Detect tracking and analytics components.

Example: Multiple advertising trackers identified.

Concept: Trackers impact user privacy.

How it finds:

- Matches known tracker library signatures.

How it marks result:

- Tracker found → **privacy notice**

5.4 Hardcoded Secrets

Logic: Detect embedded secrets.

Example: API token hardcoded in source code.

Concept: Secrets should never be embedded in apps.

How it finds:

- Searches for secret-like patterns.
- Entropy checks.

How it marks result:

- Secret detected → **high risk**

5.5 Strings

Logic: Extract readable text from the app.

Example: Debug messages reveal internal logic.

Concept: Strings can leak sensitive information.

How it finds:

- Extracts all readable text.
- Filters interesting content.

How it marks result:

- Listed for analyst review.

6. Application Components

6.1 Components

Logic: List all internal app building blocks.

Example: Activities, services, receivers identified.

Concept: Component structure defines app behavior.

How it finds:

- Parses component declarations.

How it marks result:

- Informational listing.

6.2 Activities

Logic: Identify user-facing screens.

Example: Payment and login screens detected.

Concept: Activities represent user interaction points.

How it finds:

- Reads screen definitions.

How it marks result:

- Listed with exposure status.

6.3 Services

Logic: Identify background-running tasks.

Example: Service runs continuously after app closes.

Concept: Background services may impact privacy.

How it finds:

- Detects background task definitions.

How it marks result:

- Long-running service → **privacy concern**

6.4 Receivers

Logic: Identify event listeners.

Example: Receiver triggers when SMS is received.

Concept: Receivers react to system events.

How it finds:

- Reads event listeners.

How it marks result:

- Sensitive trigger → **warning**

6.5 Providers

Logic: Identify data-sharing interfaces.

Example: Content provider exposes internal database.

Concept: Providers control data access.

How it finds:

- Detects data-sharing interfaces.

How it marks result:

- Public access → **high risk**

7. Dependencies & Inventory

7.1 Libraries

Logic: Identify third-party libraries.

Example: Outdated crypto library detected.

Concept: External code introduces external risk.

How it finds:

- Matches known library signatures.

How it marks result:

- Vulnerable version → **warning**

7.2 SBOM (Software Bill of Materials)

Logic: Create a complete dependency list.

Example: Library name, version, and license recorded.

Concept: Transparency enables risk management.

How it finds:

- Aggregates dependency data.

How it marks result:

- Inventory output (no risk).

8. File System Analysis

8.1 Files

Logic: List all files inside the app package.

Example: Backup file contains plaintext credentials.

Concept: Stored files can leak sensitive data.

How it finds:

- Iterates over all packaged files.

How it marks result:

- Sensitive file → **risk flagged**

[Web Check](#)



We give you X-Ray Vision for your Website

In just 20 seconds, you can see *what attackers already know*

Enter a URL to start

https://gosense.ai

Analyze URL

SSL Info

Subject	www.bbc.com
Issuer	GlobalSign nv-sa
Expires	14 April 2024
Renewed	12 March 2023
Serial Num	27A3771A504A5E2707...
Fingerprint	F8:8F:99:99:FC:D8:D...
Extended Key Usage	
TLS Web Server Authentication	
TLS Web Client Authentication	

DNS Records

A	151.101.64.81
AAAA	
151.101.128.81	
151.101.64.81	
151.101.192.81	
MX	
2a04:de42:600::81	
2a04:de42:400::81	
2a04:de42:200::81	
2a04:de42:181	
CNAME	
ddns1.bbc.com	
ddns8.bbc.co.uk	
ddns.bbc.com	
ddns1.bbc.co.uk	
ddns1.bbc.com	
ddns8.bbc.co.uk	
ddns8.bbc.com	
ddns1.bbc.co.uk	

Cookies

SOCS	CAABg1A2qK1Bg
expires	Tue, 06-Aug-2024 16:42:11...
path	/
domain	.google.com
SameSite	Lax
AEC	A249WE1QV0G1H6K3y7G21Bck...

DNS Server

DNS Server #1	
IP Address	104.16.132.229
DNS Support	No*
DNS Server #2	
IP Address	104.16.133.229
DNS Support	No*

* DNS Support is determined by the DNS server's response to a DNS query. Sometimes this gives false negatives, and it's also possible that the DNS server supports DNS but does not respond to DNS queries. If the DNS server does not support DNS, it may still be possible to use DNS by using a DNS proxy.

Tech Stack

React	JavaScript Framework
React	React is an open-source JavaScript library for building user interfaces or UI components. Learn more at: https://react.dev/
Gatsby	
Gatsby	Gatsby is a React-based open-source framework with performance, scalability and security built-in. Learn more at: https://www.gatsbyjs.com/
OneTrust	Cookie Compliance
OneTrust	OneTrust is a cloud-based data privacy management and consent platform. Learn more at: https://www.onetrust.com/
Cloudflare Browser Insights	Analytics, RUM
Cloudflare Browser Insights	Cloudflare Browser Insights is a tool that measures the performance of websites from the perspective of users. Learn more at: https://www.cloudflare.com/bri/
Cloudflare Bot Management	Security
Cloudflare Bot Management	Cloudflare bot management solution identifies and mitigates automated traffic to protect websites from bad bots. Learn more at: https://www.cloudflare.com/bot-management/
Priority Hints	Performance
Priority Hints	Priority Hints exposes a mechanism for developers to signal a relative priority for resources to consider when fetching resources.
Pages	
Pages	
Last Modified	16 October 2018
Change Frequency	monthly
Priority	1.00
More	

Ready To Get Started?

With over **30 supported checks** you can view and analyse key website information in an instant

- ✓ Archive History
- ✓ Block List Check
- ✓ Carbon Footprint
- ✓ Cookies
- ✓ DNS Server
- ✓ DNS Records
- ✓ DNSSEC
- ✓ Site Features
- ✓ Firewall Types
- ✓ Get IP Address
- ✓ Headers
- ✓ HSTS
- ✓ HTTP Security
- ✓ Linked Pages
- ✓ Mail Config
- ✓ Open Ports
- ✓ Quality Check
- ✓ Global Rank
- ✓ Redirects
- ✓ Robots.txt
- ✓ Screenshot
- ✓ Security.txt
- ✓ Sitemap
- ✓ Social Tags
- ✓ SSL Certificate
- ✓ Uptime Status
- ✓ Tech Stack
- ✓ Known Threats
- ✓ TLS Version
- ✓ Trace Route
- ✓ TXT Records
- ✓ Whois Lookup
- ➕ More

View on GitHub

Deploy your Own

Use the API

Sponsored by Terminal Trove

The \$HOME of all things terminal.
Get updates on the latest CLI/TUI tools via the [Terminal Trove Newsletter](#)

Visit Terminal Trove

X-Content-Type-Options

No

X-Frame-Options

No

X-XSS-Protection

No

DNS Server

DNS Server #1

IP Address

15.207.2.187

Hostname

ec2-15-207-2-187.ap-south...

DoH Support

No*

DNS Server #2

IP Address

15.206.120.180

Hostname

ec2-15-206-120-180.ap-sou...

DoH Support

No*

* DoH Support is determined by the DNS server's response to a DoH query. Sometimes this gives false negatives, and it's also possible that the DNS server supports DoH but does not respond to DoH queries. If the DNS server does not support DoH, it may still be possible to use DoH by using a DoH proxy.

HSTS Check

HSTS Enabled?

No

Site does not serve any HSTS headers.

Linked Pages

Summary

Internal Link Count

22

External Link Count

4

Internal Links

External Links

TXT Records

v

spf1 include:_spf.google.com -all

yiltyy0xku

google-site-verification

K00Rd0bzFCEucIJ2Fwu0frdXyEoSudXy00JRvwmhWp8

Firewall

No*

*The domain may be protected with a proprietary or custom WAF which we were unable to identify automatically

Threats

Phishing Status

No Phishing Found

Crawl Rules

User-agent

*

Disallow

/

Disallow

/cgi-bin/

User-agent

Googlebot

Allow

/

Block Lists

AdGuard

No Not Blocked

AdGuard Family

No Not Blocked

CleanBrowsing Adult

No Not Blocked

CleanBrowsing Family

No Not Blocked

CleanBrowsing Security

No Not Blocked

CloudFlare

No Not Blocked

CloudFlare Family

No Not Blocked

Comodo Secure

No Not Blocked

Google DNS

No Not Blocked

Neustar Family

No Not Blocked

Neustar Protection

No Not Blocked

Norton Family

No Not Blocked

OpenDNS

No Not Blocked

OpenDNS Family

No Not Blocked

Quad9

No Not Blocked

Yandex Family

No Not Blocked

Yandex Safe

No Not Blocked

DNSKEY

DNSKEY - Present?

No

DS

DS - Present?

No

RRSIG

RRSIG - Present?

No

Redirects

Followed 1 redirect when contacting host

https://gosense.ai

Open Ports

80

443

Unable to establish connections to:

20, 21, 22, 23, 25, 53, 67, 68, 69, 110, 119, 123, 143, 156, 161, 162, 179, 194, 389, 587, 993, 995, 3000, 3306, 3389, 5060, 5900, 8080, 8080, 8888

Pages

/

/rbi

/npci

/sebi

/soc

/credit-underwriting

/protect-upi-accounts

/device-and-sim-binding

/data-leak-protection

/remote-access-trojans

/browser-bot-detection

/rasp

/offlinesdk

/source-code-obfuscation

/fake-appsblocker

/bot-detection

/location-spoofing

/multi-accounting

/security-anomalies

/blog

/terms-and-condition

/privacy-policy

View / Download Raw Data

Download Results

View Results

These are the raw results generated from your URL, and in JSON format. You can import these into your own program, for further analysis.

External Tools for Further Research

Hudson Rock

hudsonrock.com

Identify Infostealer infection data related to domains and emails

SSL Labs Test

ssllabs.com

Analyzes the SSL configuration of a server and grades it

Virus Total

virustotal.com

Checks a URL against multiple antivirus engines

Shodan

shodan.io

Search engine for Internet-connected devices

Archive

archive.org

View previous versions of a site via the Internet Archive

URLScan

urlscan.io

Scans a URL and provides information about the page

Sucuri SiteCheck

sitecheck.sucuri.net

Checks a URL against blacklists and known threats

Domain Tools

whois.domaintools.com

Run a WhoIs lookup on a domain

NS Lookup

nslookup.io

View DNS records for a domain

DNS Checker

dnschecker.org

Check global DNS propagation across multiple servers

Censys

search.censys.io

Lookup hosts associated with a domain

Page Speed Insights

developers.google.com

Checks the performance, accessibility and SEO of a page on mobile + desktop

Built With

builtwith.com

View the tech stack of a website

DNS Dumpster

dnsdumpster.com

DNS recon tool, to map out a domain from it's DNS records

BGP Tools

bgp.tools

View realtime BGP data for any ASN, Prefix or DNS

Similar Web

similarweb.com

View approx traffic and engagement stats for a website

Blacklist Checker

blacklistchecker.com

Cloudflare Radar

radar.cloudflare.com

Mozilla HTTP Observatory

developer.mozilla.org

AbuseIPDB

abuseipdb.com

Web Check

gosense.ai

23 jobs successful 3 jobs skipped 11 jobs failed

Finished in 11600 ms

Show Details

get-ip (success). Took 272 ms

location (success). Took 329 ms

ssl (success). Took 485 ms

domain (success). Took 384 ms

quality (error). Took 247 ms

tech-stack (error). Took 257 ms

server-info (success). Took 725 ms

cookies (success). Took 1102 ms

headers (success). Took 1860 ms

dns (success). Took 1448 ms

hosts (success). Took 725 ms

http-security (success). Took 1131 ms

social-tags (success). Took 1969 ms

trace-route (error). Took 26787 ms

security-txt (success). Took 1201 ms

dns-server (success). Took 515 ms

firewall (success). Took 1084 ms

dnssec (success). Took 1448 ms

hsts (success). Took 773 ms

threats (success). Took 1223 ms

mail-config (error). Took 25427 ms

archives (skipped). Took 6331 ms

rank (skipped). Took 584 ms

screenshot (error). Took 315 ms

tls-cipher-suites (error). Took 314 ms

tls-security-config (error). Took 314 ms

tls-client-support (error). Took 314 ms

rediracts (success). Took 1131 ms

linked-pages (success). Took 1995 ms

robots-txt (success). Took 665 ms

status (error). Took 1332 ms

ports (success). Took 1759 ms

txt-records (success). Took 314 ms

block-lists (success). Took 276 ms

features (error). Took 268 ms

sitemap (success). Took 671 ms

carbon (error). Took 11096 ms

Check the browser console for logs and more info

It's normal for some jobs to fail, either because the host doesn't return the required info, or restrictions in the lambda function, or hitting an API limit.

Learn More about Web-Check

Dismiss

Server Location

City400072, Mumbai, Maharasht...

CountryIndia

TimezoneAsia/Kolkata

Languagesen-IN,hi,bn,te,mr,ta,ur,g...

CurrencyRupee (INR)

(Amazon.com, Inc.)

Latitude: 19.0931, Longitude: 72.9049

SSL Certificate

Subject*.gosense.ai

IssuerAmazon

Expires30 October 2026

Renewed30 September 2025

Serial Num05F4A8CD8C08F311CA7...

Fingerprint88:1B:12:88:D0:D0:F...

Extended Key Usage

TLS Web Server Authentication

Domain Whois

DNS Records

A

15.207.2.187

AAAA

15.207.2.187

15.206.120.180

CNAME

ns-745.awsdns-29.net

ns-1123.awsdns-12.org

ns-1693.awsdns-19.co.uk

ns-380.awsdns-47.com

NS

yiltyy0xcu

google-site-verification=K00Rd0bzFCeucIJ2Fwu0frdXyEoSud

v=spf1 include:_spf.google.com -all

Server Info

HTTP Security

Content Security Policy

Strict Transport Policy

No

No

Social Tags

TitleSense - Identify the Good..

sense

[Web-check-results.json](#)

How to Implement in Your App ?

Android Dynamic Analyzer – Logical Design

1. Purpose of the Dynamic Analyzer

The Android Dynamic Analyzer is designed to **observe and analyze the runtime behavior of Android applications** by executing them in a controlled environment and monitoring their interactions with the operating system, network, and device resources.

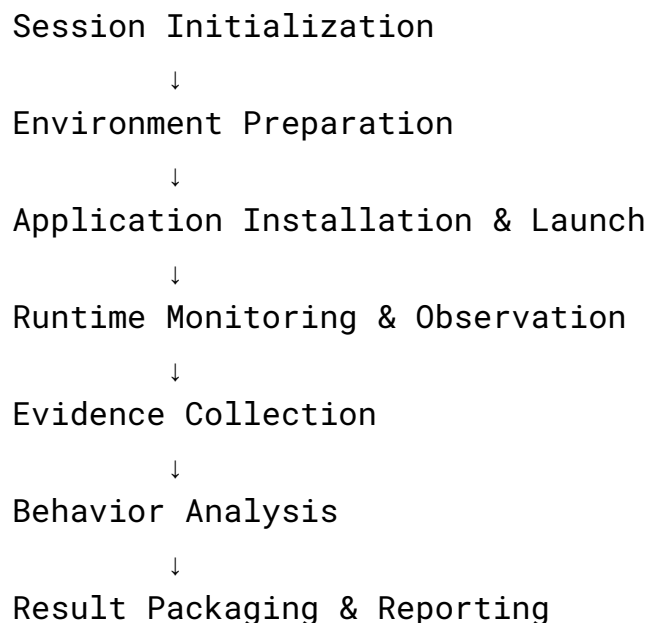
Unlike static analysis, this system focuses on **what the application actually does at runtime**.

2. Core Design Philosophy

The system is built on the following principles:

1. **Runtime-first observation**
2. **Separation of concerns** between execution, monitoring, and analysis
3. **Event-based behavior collection**
4. **Environment isolation and reproducibility**
5. **Post-execution analysis over raw log parsing**

3. High-Level Logical Flow



Each phase is logically independent and can be extended or replaced without affecting the others.

4. Logical Components and Responsibilities

4.1 Session Orchestrator

Responsibility

Controls the **entire lifecycle** of a dynamic analysis session.

Logic

- Validate analysis request
- Create a unique session context
- Coordinate all subsystems
- Handle failures and cleanup

Output

A managed execution flow that ensures correct sequencing of analysis stages.

4.2 Environment Manager

Responsibility

Prepare and validate the Android execution environment.

Logic

- Detect connected Android device or emulator
- Verify OS version compatibility
- Check required runtime capabilities
- Ensure environment readiness

Key Principle

Analysis should never begin unless the environment is explicitly validated.

4.3 Application Runtime Controller

Responsibility

Manage the target application's lifecycle during analysis.

Logic

- Install application package

- Launch application
- Track running process
- Terminate application after analysis

Key Principle

Application control is separate from behavior observation.

4.4 Network Configuration Manager

Responsibility

Enable controlled interception of network traffic.

Logic

- Initialize traffic interception service
- Configure device-level proxy routing
- Enable bidirectional communication
- Restore network state after analysis

Key Principle

Network interception must be reversible and isolated per session.

4.5 Runtime Instrumentation Engine

Responsibility

Observe sensitive runtime operations performed by the application.

Logic

- Attach to the application process
- Monitor calls related to:
 - Cryptography
 - File system access
 - Inter-process communication
 - Network APIs
- Emit structured runtime events

Key Principle

Runtime behavior should be captured as structured events, not raw logs.

4.6 Telemetry Collectors

Responsibility

Collect auxiliary runtime information.

Logic

- Capture system logs
- Monitor clipboard usage
- Observe runtime state changes
- Collect screenshots and memory artifacts

Output

A stream of contextual runtime evidence.

4.7 Evidence Aggregator

Responsibility

Consolidate runtime evidence into a unified dataset.

Logic

- Normalize collected events
- Merge data from multiple sources
- Deduplicate artifacts
- Organize evidence by category

Key Principle

Raw runtime data must be normalized before analysis.

4.8 Behavior Analysis Engine

Responsibility

Convert runtime evidence into security-relevant findings.

Logic

- Apply detection rules
- Correlate events across sources
- Identify insecure behaviors

- Classify findings by severity

Example Behaviors

- Insecure cryptographic usage
- Cleartext network transmission
- Sensitive data leakage
- Unauthorized resource access

4.9 Artifact Exporter

Responsibility

Prepare analysis outputs for consumption.

Logic

- Package logs, network traces, and artifacts
- Generate structured result files
- Support download and archival formats

Key Principle

Analysis results should be reproducible and verifiable.

5. Event-Driven Observation Model

All runtime components generate **events** rather than raw output.

Event Characteristics

- Timestamped
- Typed (network, crypto, log, file, etc.)
- Structured (JSON-like format)

Benefits

- Simplified correlation
- Improved scalability
- Clear separation between collection and analysis

6. Error Handling & Recovery Logic

- Each subsystem reports failures independently
- Session orchestrator determines continuation or termination

- Partial evidence is preserved on failure
- Environment is always restored to a clean state

7. Security & Isolation Logic

- Each analysis session is isolated
- Network configuration is reverted after execution
- Temporary artifacts are sandboxed
- No persistent modification to target application

8. Limitations (Logical)

- Anti-instrumentation techniques may reduce visibility
- Emulator-based analysis may differ from real devices
- Encrypted or obfuscated runtime behavior may limit extraction

9. Extensibility Logic

The design supports:

- Additional runtime observers
- New detection rules
- Alternative instrumentation engines
- Multi-device execution

No component assumes implementation-specific behavior.

10. Summary

The Android Dynamic Analyzer is a **modular, event-driven runtime analysis system** that:

- Executes applications in controlled environments
- Observes real runtime behavior
- Extracts structured evidence
- Produces actionable security insights

This design ensures **originality, scalability, and clarity**, while remaining independent of any specific framework or existing implementation.